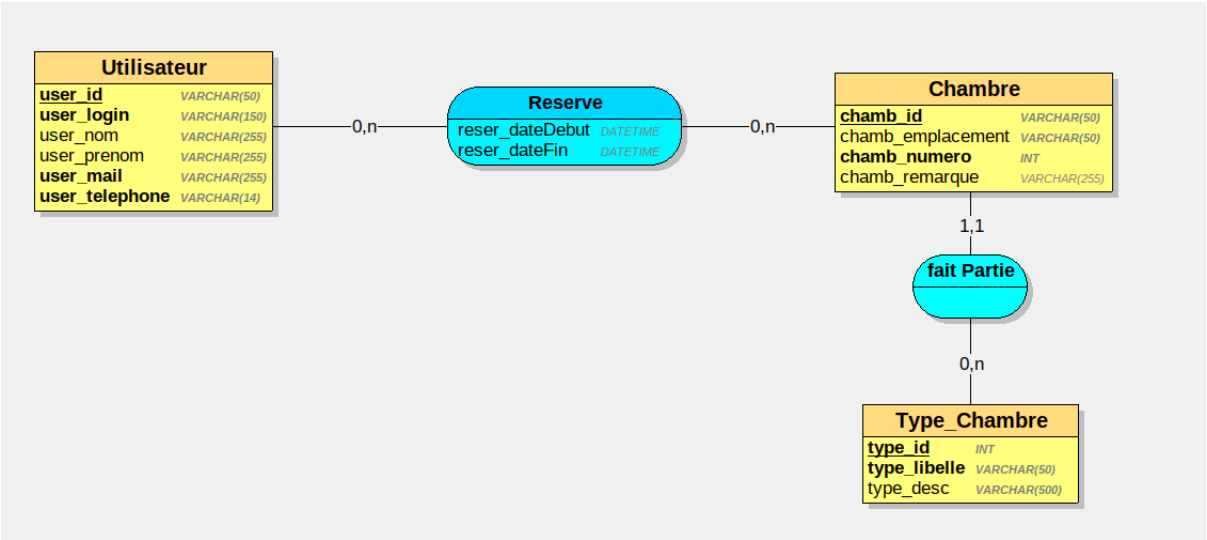
Conception de la base de donnée : 2
Modèle MCD: 2
Modèle MLD: 3
MLD Textuel : 4
Modèle MPD : 5
Organisation du projet : 7
Diagramme de Gantt: 7
Sous - Groupe : 8
Jalon: 9
Réservation : 9
1. Description du Flux Fonctionnel 9
2. Implémentation Technique (Logique Métier) 9
3. Accès aux Données (Modèles) 10
4. Interface Utilisateur (Vues) 10
Authentification : 10
1. Description du Flux Fonctionnel 10
2. Architecture des Données (Conception) 11
3. Implémentation Technique (Logique MVC) 11
A. Modèle (UtilisateurModel) 11
B. Contrôleur (Logique Métier) 11
C. Vues (Interface) 11
4. Sécurité 12
5. État d'avancement 12

Conception de la base de donnée :

Modèle MCD:



Ce schéma relationnel nous permet d'organiser et de structurer les différentes entités nécessaires au bon fonctionnement du système.

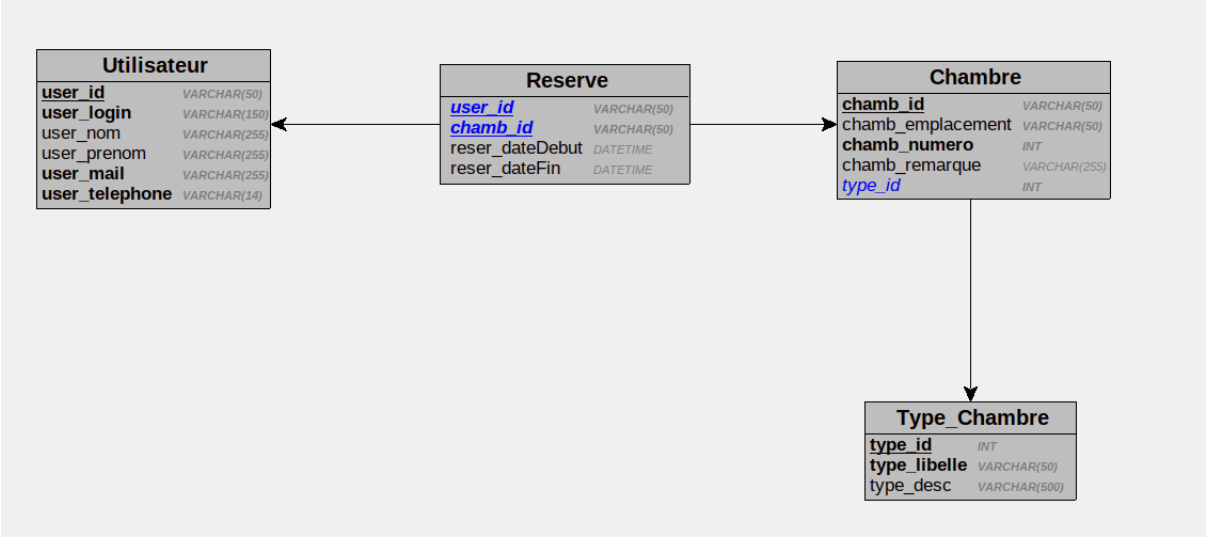
Nous avons tout d'abord défini une entité **Utilisateur**, qui regroupe les informations personnelles des clients, telles que l'identifiant, le login, le nom, le prénom, l'adresse mail et le numéro de téléphone. Ces utilisateurs **ont la possibilité d'effectuer des réservations**, ce qui est géré par l'entité **Réserve**. Chaque réservation est caractérisée par une date de début et une date de fin. La table **Réserve** est une table d'association entre la table **Utilisateur** et la table **Chambre**, représentant une relation "many-to-many". Par conséquent, la clé primaire de chaque table va migrer dans la table **Réserve** pour former une clé composite. Un même utilisateur peut avoir plusieurs réservations à différents moments.

Les réservations **sont associées à des chambres**, représentées par l'entité **Chambre**, dans laquelle nous avons inclus l'identifiant de la chambre, son emplacement dans l'établissement, son numéro ainsi qu'une éventuelle remarque (par exemple, des spécificités ou des limitations liées à la chambre).

Chaque chambre appartient à un type de chambre, défini dans l'entité **Type_Chambre**. Ce type permet de regrouper les chambres selon leur catégorie (par exemple, logement, chambre double, chambre triple, etc.) grâce à un libellé et une description détaillée. Dans ce cas, la clé primaire de l'entité **Type_Chambre** migre dans l'entité **Chambre** pour identifier à quel type de chambre chaque chambre appartient.

Ce modèle, que nous avons élaboré en groupe, assure une bonne cohérence des données et permet une gestion fluide des réservations. Il est conçu pour être évolutif, clair et adapté à un usage réel dans le cadre d'un système de réservation d'hôtel.

Modèle MLD:



Utilisateur ↔ Réserve : La relation entre l'**Utilisateur** et la **Réserve** est de type "0,n" : un utilisateur peut ne pas réserver, ou alors réservez plusieurs chambres.

Chambre ↔ Réserve : La relation entre la **Chambre** et la **Réserve** est aussi de type "0,n" : une chambre peut ne pas être réservée ou alors elle peut être réservée une ou plusieurs fois.

Type_Chambre ↔ Chambre : La relation entre **Type_Chambre** et **Chambre** est de type "1,n" : un type de chambre peut être lié à plusieurs chambres, mais chaque chambre appartient à un seul type de chambre.

MLD Textuel :

Utilisateur = (user_id VARCHAR(50), user_login VARCHAR(150), user_nom VARCHAR(255), user_prenom VARCHAR(255), user_mail VARCHAR(255), user_telephone VARCHAR(14),user_password VARCHAR(255),user_role INT);

Type_Chambre = (type_id INT, type_libelle VARCHAR(50), type_desc VARCHAR(500));

Chambre = (chamb_id VARCHAR(50), chamb_emplacement VARCHAR(50), chamb_numero INT, chamb_remarque VARCHAR(255), #type_id);

Reserve = (#user_id, #chamb_id, reser_dateDebut DATETIME, reser_dateFin DATETIME);

Modèle MPD :

```
CREATE TABLE Utilisateur(
  user_id VARCHAR(50),
  user_login VARCHAR(150) NOT NULL,
  user_nom VARCHAR(255) NOT NULL,
  user_prenom VARCHAR(255) NOT NULL,
  user_mail VARCHAR(255) NOT NULL,
  user_telephone VARCHAR(14) NOT NULL,
  PRIMARY KEY(user_id),
  UNIQUE(user_login),
  UNIQUE(user_mail),
  UNIQUE(user_telephone)
);
```

```
CREATE TABLE Type_Chambre(
  type_id INT,
  type_libelle VARCHAR(50) NOT NULL,
  type_desc VARCHAR(500) NOT NULL,
  PRIMARY KEY(type_id),
  UNIQUE(type_libelle)
);
```

```
CREATE TABLE Chambre(
  chamb_id VARCHAR(50),
  chamb_emplacement VARCHAR(50) NOT NULL,
  chamb_numero INT NOT NULL,
  chamb_remarque VARCHAR(255),
  type_id INT NOT NULL,
  PRIMARY KEY(chamb_id),
  UNIQUE(chamb_numero),
  FOREIGN KEY(type_id) REFERENCES Type_Chambre(type_id)
);
```

```
CREATE TABLE Reserve(
  user_id VARCHAR(50),
  chamb_id VARCHAR(50),
  reser_dateDebut DATETIME NOT NULL,
  reser_dateFin DATETIME NOT NULL,
  PRIMARY KEY(user_id, chamb_id),
  FOREIGN KEY(user_id) REFERENCES Utilisateur(user_id),
  FOREIGN KEY(chamb_id) REFERENCES Chambre(chamb_id)
);
```

La table **Utilisateur** contient toutes les informations personnelles des clients, telles que l'identifiant, le login, le nom, le prénom, l'adresse mail et le numéro de téléphone. L'identifiant user_id sert de clé primaire, tandis que le login, l'adresse mail et le téléphone sont uniques pour éviter les doublons. La table **Type_Chambre** regroupe les catégories de chambres disponibles dans l'hôtel, avec un identifiant type_id comme clé primaire, un libellé unique et une description détaillée.

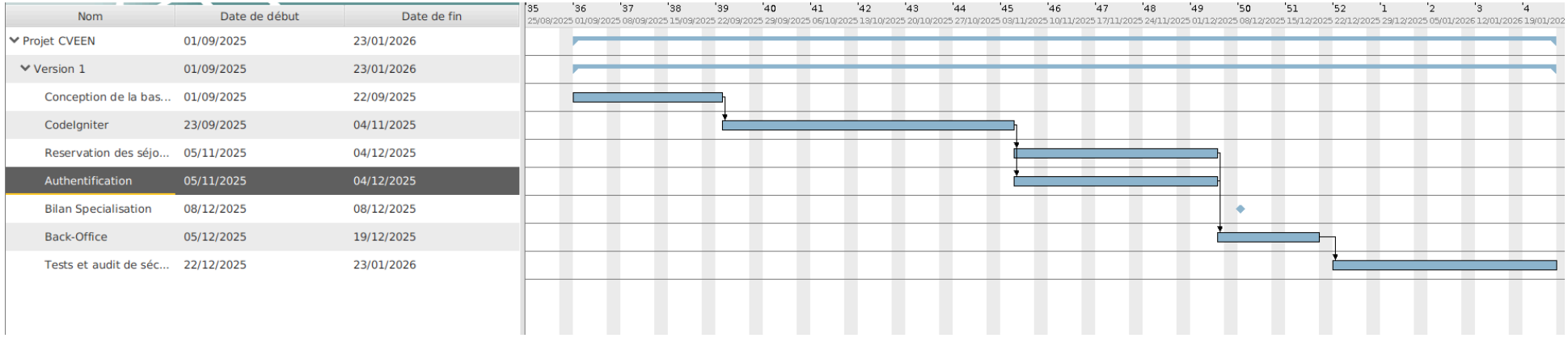
La table **Chambre** contient les informations spécifiques de chaque chambre : identifiant, emplacement, numéro unique et éventuelles remarques. Chaque chambre est liée à un type de chambre via la clé étrangère type_id, ce qui traduit une relation 1:N entre Type_Chambre et Chambre.

Enfin, la table **Reserve** permet de gérer les réservations en associant les utilisateurs et les chambres. Elle contient les identifiants user_id et chamb_id ainsi que les dates de début et de fin de la réservation. La clé primaire est composée de user_id, chamb_id et reser_dateDebut afin de permettre à un même utilisateur de réserver la même chambre à différentes périodes. Les clés étrangères assurent l'intégrité référentielle avec les tables Utilisateur et Chambre. Ce MPD garantit ainsi la cohérence et l'intégrité des données tout en permettant une gestion flexible et efficace des réservations dans le système hôtelier.

Organisation du projet :

Diagramme de Gantt:

Afin de représenter la répartition temporelle des tâches et le déroulement global du projet, un diagramme de Gantt a été élaboré.

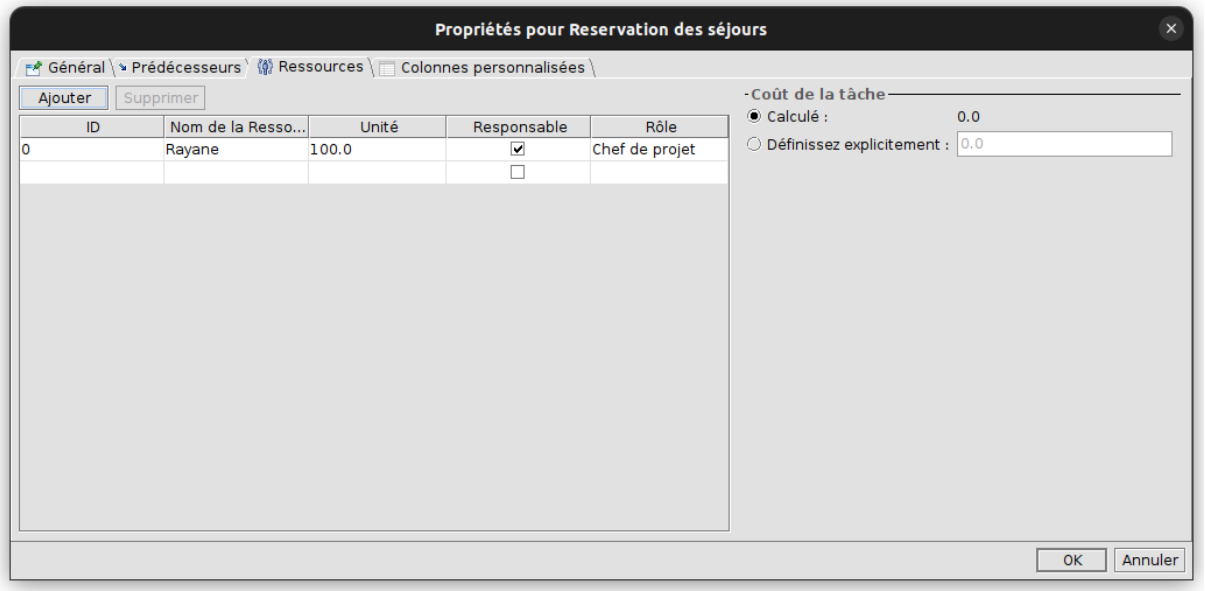


Nom	Date de début	Date de fin
Projet CVEEN	01/09/2025	23/01/2026
Version 1	01/09/2025	23/01/2026
Conception de la bas...	01/09/2025	22/09/2025
CodeIgniter	23/09/2025	04/11/2025
Reservation des séjo...	05/11/2025	04/12/2025
Authentification	05/11/2025	04/12/2025
Bilan Specialisation	08/12/2025	08/12/2025
Back-Office	05/12/2025	19/12/2025
Tests et audit de séc...	22/12/2025	23/01/2026

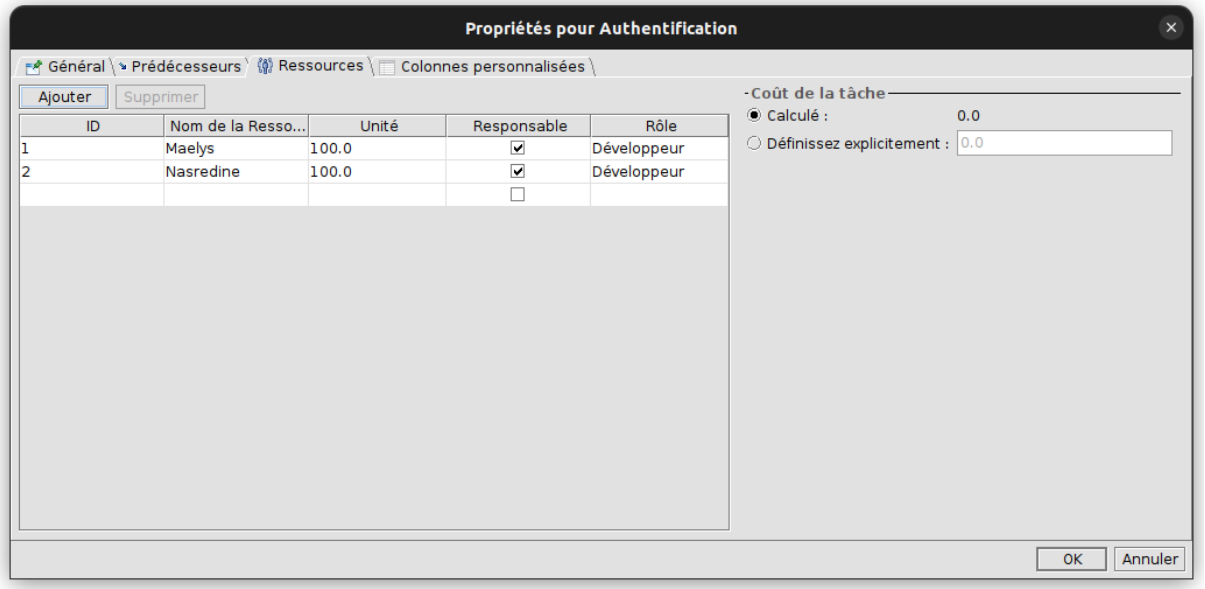
Le diagramme de Gantt ci-dessous présente la planification du projet.
Il met en évidence les différentes étapes : Conception de la bdd, Réservation des séjours , Authentification…, ainsi que leurs durées respectives.
Ce planning permet de suivre l’avancement des travaux et d’ajuster les priorités en fonction des contraintes rencontrées.

Sous - Groupe :

La tâche de **réservation des séjours** sera prise en charge par le sous-groupe Rayane WATTEEUW, qui sera responsable de la mise en place des fonctionnalités permettant aux utilisateurs d'effectuer, consulter et gérer leurs réservations.



La tâche d'authentification sera réalisée par le sous-groupe Maelys LE BRUN & Nasredine BECHITIBAT, chargés de développer les fonctionnalités de **connexion (login)** et **d'inscription (register)** des utilisateurs dans le cadre du projet.



Jalon:

Un **jalon** a été positionné à la fin de ces deux tâches comme point de **go/no go**, afin de vérifier le bon fonctionnement et la stabilité des deux sous-applications avant de passer à l'intégration et aux phases suivantes du développement. Ce jalon permettra de valider la cohérence entre les modules, d'identifier d'éventuels problèmes d'interfaçage afin d'assurer une base solide pour la suite du projet.

Réservation :

La fonctionnalité de réservation constitue le cœur du système CVVEN. Ce module a donc été réalisé par Rayane WATTEEUW. Elle permet de concrétiser la transaction entre l'hôtel et le client en associant une chambre spécifique à un utilisateur pour une période donnée. Ce module a été développé en respectant strictement l'architecture MVC (Modèle-Vue-Contrôleur) imposée par le framework CodeIgniter 4.

1. Description du Flux Fonctionnel

Le processus de réservation a été conçu pour être le plus fluide possible. Il s'initie depuis la page de détail d'une chambre. L'utilisateur, qu'il soit déjà client ou simple prospect, remplit un formulaire unique comprenant ses informations personnelles (Nom, Prénom, Email, Téléphone) et les dates de son séjour. À la validation du formulaire, le système traite la demande de manière synchrone : il vérifie la disponibilité, gère l'identité du client et enregistre la réservation en base de données avant de confirmer l'action à l'utilisateur.

2. Implémentation Technique (Logique Métier)

La logique métier est centralisée dans le contrôleur Hotel.php, spécifiquement au sein de la méthode reserver().

Algorithme de traitement : Le traitement suit une séquence rigoureuse pour garantir l'intégrité des données :

1. **Validation des Entrées :** Avant tout traitement, les données soumises sont filtrées par le service de validation de CodeIgniter. Des règles strictes sont appliquées : les dates doivent être valides (valid_date), l'email conforme (valid_email) et tous les champs requis présents.
2. **Vérification de Cohérence :** Une règle métier conditionnelle s'assure que la date de départ est strictement postérieure à la date d'arrivée. En cas d'incohérence, l'utilisateur est redirigé vers le formulaire avec un message d'erreur explicite.
3. **Gestion Dynamique des Utilisateurs (Prospects) :**
    - Le système vérifie d'abord si l'adresse email saisie correspond à un utilisateur existant via UtilisateurModel.
    - Si l'utilisateur est inconnu, le système crée un compte "à la volée". Un identifiant unique est généré automatiquement (préfixe USR suivi d'une chaîne unique) ainsi

qu'un login basé sur le nom et le prénom, permettant de transformer instantanément un prospect en client enregistré sans procédure d'inscription préalable lourde.

4. **Persistance des Données :** Une fois l'utilisateur identifié (ou créé), la réservation est insérée dans la table Reserve via ReserveModel. Cette table d'association stocke le lien entre l'user_id, le chamb_id et les dates.

3. Accès aux Données (Modèles)

L'interaction avec la base de données s'appuie sur deux modèles distincts pour respecter la séparation des responsabilités :

- **UtilisateurModel** : Gère la création et la récupération des fiches clients. Il garantit que les informations de contact (email, téléphone) sont correctement stockées.
- **ReserveModel** : Gère exclusivement les enregistrements de séjours. Il définit les champs autorisés (allowedFields) pour protéger la base contre les injections de masse et valide le format des dates au niveau du modèle.

4. Interface Utilisateur (Vues)

L'interface de réservation est intégrée directement dans la vue hotel/detail.php.

- **Formulaire Contextuel :** Le formulaire est pré-rempli avec l'identifiant de la chambre (chamb_id en champ caché) pour éviter toute erreur de contexte.
- **Sécurité CSRF :** L'inclusion de la directive csrf_field() dans le formulaire protège l'application contre les attaques par falsification de requête inter-sites.
- **Feedback Utilisateur :** En cas de succès, une vue dédiée hotel/success.php est affichée pour confirmer la prise en compte de la demande. En cas d'erreur (disponibilité, validation), les messages sont remontés via la session flash (session('errors')) et affichés en rouge pour alerter l'utilisateur.

Authentification :

Le module d'authentification a été identifié comme une fonctionnalité critique, attribuée au sous-groupe technique composé de Maelys LE BRUN et Nasredine BECHITIBAT. Il vise à sécuriser l'accès à l'application et à permettre aux utilisateurs de retrouver leur historique de réservations.

1. Description du Flux Fonctionnel

Le système d'authentification distingue deux parcours utilisateurs :

- **Inscription (Register)** : Permet à un nouveau visiteur de créer un compte pérenne. Contrairement à la création de "prospects" à la volée utilisée dans le module Réservation, l'inscription requiert la définition d'un mot de passe sécurisé.
- **Connexion (Login)** : L'utilisateur s'identifie via son login (ou email) et son mot de passe pour accéder à son espace personnel.

2. Architecture des Données (Conception)

L'analyse du Modèle Logique de Données (MLD) textuel révèle la structure prévue pour supporter l'authentification :

- **Table Utilisateur** : Outre les informations d'identité (nom, prénom), le schéma inclut deux champs cruciaux pour ce module :
  - user_password (VARCHAR 255) : Destiné à stocker le mot de passe haché.
  - user_role (INT) : Permet de gérer les niveaux d'autorisation (ex: Client vs Administrateur).

Point de vigilance technique : Il a été relevé que le script SQL de création de table actuel (CREATE TABLE Utilisateur) n'inclut pas encore ces deux colonnes. Une migration de la base de données sera nécessaire pour aligner le MPD (Modèle Physique) sur le MLD (Modèle Logique) et rendre le login fonctionnel.

3. Implémentation Technique (Logique MVC)

L'implémentation suit le pattern MVC du projet :

A. Modèle (UtilisateurModel)

Le modèle existant devra être étendu pour :

- Ajouter user_password et user_role dans la liste des champs autorisés ($allowedFields).
- Intégrer des *Callbacks* (ex: beforeInsert, beforeUpdate) pour hacher automatiquement les mots de passe avant l'enregistrement en base, garantissant qu'aucun mot de passe ne soit stocké en clair.

B. Contrôleur (Logique Métier)

Le traitement sera assuré par un contrôleur dédié (ex: AuthController), responsable de :

1. **Validation** : Vérifier que les champs obligatoires sont remplis.
2. **Vérification des identifiants** :
   - Recherche de l'utilisateur par son login/email.
   - Comparaison du mot de passe saisi avec le hash en base via password_verify().
3. **Gestion de Session** : En cas de succès, initialisation de la session CodeIgniter avec les données utilisateur (user_id, user_role, is_logged_in).

C. Vues (Interface)

Les vues dédiées (auth/login.php, auth/register.php) devront intégrer les mécanismes de sécurité standards :

- Protection **CSRF** (csrf_field()) pour sécuriser les formulaires.

- Affichage des messages d'erreur via les données "Flash" de la session (similaire au module Réservation).

4. Sécurité

Le module repose sur les standards de sécurité de CodeIgniter 4 :

- **Hachage** : Utilisation des algorithmes robustes (Bcrypt ou Argon2) par défaut dans PHP.
- **Isolation** : Les données sensibles (mots de passe) ne sont jamais transmises aux vues.
- **Session** : Régénération de l'ID de session (session()->regenerate()) lors de la connexion pour prévenir les attaques par fixation de session.

5. État d'avancement

Selon le planning du projet (Diagramme de Gantt), cette tâche est planifiée en parallèle du module de réservation. Un jalon de validation ("Go/No Go") est positionné à la fin du développement pour valider l'intégration entre la gestion des comptes (Authentification) et la gestion des séjours (Réservation).